

RB-Y1 팔레타이징 — 슬롯 1 위치를 어떻게 맞추는가

대상: 슬롯 1 정렬 경로 (pallet.py live --auto-palletize-slot1) | 근거 코드: pallet_servo.py, pallet_runtime.py, configs/rby1m_v1_2_pallet_slot1_nominal.json | 2026-07-30

한 문장 요약

카메라로 지금 보이는 팔레트 중앙 구멍의 중심을, 작업자가 예전에 손으로 맞춰둔 좌표와 비교해서 그 차이가 0 이 되도록 모바일 베이스를 움직입니다. 차이가 충분히 작아진 상태가 일정 시간 유지되고, 실제 바퀴가 멈춘 것까지 확인되면 "도착"으로 판정합니다.

1. 큰 그림



그림 1. 로봇 base 좌표에서 본 팔레트와 기준점. 초록 화살표를 0 으로 만드는 것이 정렬의 전부다.

정렬에 등장하는 것은 딱 세 가지입니다.

이름	무엇인가	어디서 오는가
현재 관측	지금 카메라에 보이는 팔레트 적층의 중심과 방향	D435 depth 로 적층 윗면 평면을 찾고, 중앙 홀 또는 외곽 L 코너를 실측 치수 모델에 맞춰 계산
시연 기준점	"여기가 슬롯 1 을 놓을 수 있는 자리"라고 기록해 둔 값	설정 파일에 박혀 있는 고정값. 작업자가 눈으로 맞춘 자세에서 16 프레임 평균
오차	현재 관측 - 시연 기준점 (x, y, yaw 세 개)	매 카메라 프레임마다 새로 계산

정확히 무엇을 무엇에 맞추는가 (자주 오해하는 지점)

맞추는 대상은 중앙 네모 구멍의 중심 (x, y)와 그 방향 (yaw), 세 개입니다.
목표는 설정에 적힌 [0.865000, 0.139523] m, -90 도입니다.
즉 "추정한 구멍 중심이 지정한 좌표에 오도록" 맞추는 것이 맞습니다.

다만 세 가지를 구분해야 합니다.

(1) 움직이는 것은 팔레트가 아니라 로봇입니다. 두 값 모두 base 좌표, 즉 로봇에 붙어 있는 좌표계로 표현됩니다. 팔레트는 가만히 있고 로봇이 움직이므로, "구멍 중심의 base 좌표를 [0.865, 0.1395]로 만든다"는 것은 실제로는 로봇이 팔레트에 대해 정해진 자리로 이동한다는 뜻입니다.

(2) 서보가 쫓는 것은 구멍 중심이고, 슬롯 1 지점이 아닙니다. 슬롯 1 은 구멍 중심에서 고정 오프셋(+0.20175 m, +0.128 m)만큼 떨어진 곳으로 계산됩니다(그림 1 점선). 서보는 그 오프셋을 쓰지 않습니다. 구멍 중심만 기준점에 맞춥니다.

그래도 되는 이유는, 기준점 자체가 "이 자리에 서면 고정 ready 자세의 양팔이 슬롯 1 위에 온다"는 관계를 작업자가 시연으로 확인한 값이기 때문입니다. 팔은 정렬 중 고정된 자세로 있습니다.

(3) 구멍이 안 보일 때도 맞추는 양은 같습니다. 멀어서 중앙 구멍이 보이지 않으면 외곽 L 코너로 같은 구멍 중심을 역산해 씁니다. 관측 방법만 바뀌고 비교 대상은 언제나 구멍 중심입니다. 그래서 소스가 바뀌면 필터를 재시작합니다.

한 줄로: 추정된 중앙 구멍 중심 (x, y)과 방향(yaw)이 지정한 [0.865, 0.139523] m, -90 도가 되도록

모바일 베이스를 움직입니다. 팔은 그동안 고정입니다.

시연 기준점의 실제 값

```
"slot1_hole_reference": {
  "center_base_xy_m": [0.865000, 0.139523],    <- 목표 위치 (base 좌표, 미터)
  "yaw_base_deg": -90.0,                      <- 목표 방향
  "source_selection": "complete_hole_dwell_complete_frames",
  "source_frame_count": 16,                   <- 16 프레임 평균
  "center_std_xy_m": [0.000604, 0.000728],    <- 프레임 간 산포 0.6~0.7 mm
  "calibration_status": "nominal_unverified_operator_demonstrated"
}
```

중요 — 이건 "정답 위치"가 아닙니다.

작업자가 눈대중으로 맞춘 자리를 기록한 값입니다. 산포 0.6 mm 는 같은 자리를 얼마나 일관되게

읽는지(재현성)를 뜻하고, 그 자리가 실제 슬롯 1 의 정중앙인지(정확도)는 검증되지 않았습니다. 설정의

상태 표기도 nominal_unverified_operator_demonstrated 입니다.

2. 팔레트를 어떻게 "본다"는 것인가

RGB 사각형 검출이 아닙니다. 깊이 카메라로 금속적 평면과 실측 치수를 맞춥니다. 순서는 이렇습니다.

1. depth 로 적층 윗면 평면(stack top plane)을 찾는다
2. 그 평면 위 점들을 위에서 내려다본 격자로 만든다
3. 중앙 구멍이 온전히 보이면 그 구멍을 실측 148×149 mm 모델에 맞춘다 → 중심과 방향을 직접 얻음
4. 멀어서 구멍이 안 보이면, 보이는 가까운 쪽 외곽선과 오른쪽 측면 외곽선이 만드는 ㄱ자 코너(L 코너)를 실측 660×658 mm 외곽 모델에 맞춰 중심을 역산한다

관측 소스	로그 표기	권한
중앙 홀 직접 관측	<code>complete_hole</code>	정렬 전체 (x, y, yaw)
외곽 L 코너 (엄격)	<code>fixed_outer_l_corner</code>	정렬 전체 (x, y, yaw)
외곽 edge (느슨)	<code>coarse</code>	전진만. 좌우·회전·후진·하강·릴리스 금지

느슨한 근거에 작은 전진만 허용하는 이유는, 그 정보로는 좌우 위치를 신뢰할 수 없기 때문입니다. 전진 총량도 0.15 m 로 묶여 있고 속도는 0.030 m/s 입니다.

u_right 와 v_far — 팔레트 기준 방향

팔레트에는 자체 방향이 있습니다. `u_right` 는 로봇이 봤을 때 오른쪽, `v_far` 는 로봇에서 먼 쪽입니다. 슬롯 1 의 위치는 적층 중심에서 +0.20175 m(`v_far`), +0.128 m(`u_right`)만큼 떨어진 곳으로 정의됩니다(그림 1 의 점선).

이 방향을 뒤집으면 슬롯이 엉뚱한 곳으로 가므로, 코드는 `axis_branch = "image_right"`와 "`v_far` 는 base +X 방향"을 계약으로 고정합니다. 관측의 축 방향이 기준과 다르면 그 관측은 거부됩니다.

3. 오차에서 속도를 만드는 방법

세 방향을 순서대로 맞추지 않습니다. 20 Hz 로 v_x , v_y , w_z 를 동시에 계산합니다. 앞뒤 먼저 맞추고 좌우를 맞추는 식이 아닙니다.

$$\begin{aligned}w_z &= 0.8 \times \text{yaw 오차} \\v_x &= 0.7 \times x \text{ 오차} + (\text{관측점 } y) \times w_z \\v_y &= 0.7 \times y \text{ 오차} - (\text{관측점 } x) \times w_z\end{aligned}$$

뒤에 붙은 항이 핵심입니다. 베이스가 회전하면 관측 대상이 시야에서 옆으로 밀려나므로, 회전이 만들어낸 이동분을 미리 상쇄해 줍니다. 이것이 없으면 회전할 때 위치가 흐트러지고 서로를 쫓아다니게 됩니다.

속도에 걸리는 세 겹의 제한

제한	값	이유
속도 상한	직선 0.05 m/s, 회전 0.08 rad/s	상한을 넘으면 세 성분을 같은 비율로 줄인다. 방향이 변하지 않게 하려고
가속 제한	0.12 m/s ² , 0.18 rad/s ²	급출발·급정지로 든 상자가 흔들리지 않게
하드 리밋	0.08 m/s, 0.10 rad/s	설정으로도 넘을 수 없는 코드 내 상한

4. 잘못된 관측을 걸러내는 장치

장치	기준	동작
중앙값 필터	3 프레임	튀는 값 1 개는 결과에 영향을 주지 않음. 3 장이 모이기 전에는 정지
점프 거부	30 mm, 15 도	한 프레임에 이만큼 튀면 그 관측을 버린다
재시딩	3 프레임 연속	단, 튕 자리에서 3 프레임 연속 일관되면 "실제로 옮겨졌다"로 보고 새로 시작
오래됨	0.30 초	이보다 오래된 관측이면 정지
시간 역행	—	타임스탬프가 뒤로 가면 즉시 FAULT
보정량 한계	0.25 m	오차가 이보다 크면 뭔가 잘못된 것으로 보고 FAULT
시작 yaw 한계	15 도	처음부터 이보다 많이 틀어져 있으면 시작하지 않음

소스 변경

—

홀 ↔ L 코너로 근거가 바뀌면 필터를 재시작

공통 원칙: 확실하지 않으면 움직이지 않습니다. 억지로 추정해서 움직이는 경우는 없습니다.

5. "도착"의 판정 — 다섯 관문



그림 2. (a) 들어가는 문턱과 나가는 문턱이 다르다. (b) 프레임 수와 시간을 둘 다 채워야 한다.

관문 1 — 두 값이 동시에 안에 있어야

필터 통과값 하나만 보지 않습니다. raw 관측과 필터값이 둘 다 범위 안에 있어야 합니다. 한쪽만 좋으면 통과하지 않습니다.

진입 (inner) 유지 (outer)

위치 오차 10 mm 이내 15 mm 이내

yaw 오차 3 도 이내 5 도 이내

들어가는 문턱은 좁고 나가는 문턱은 넓습니다(히스테리시스). 문턱 근처에서 미세하게 떨릴 때 도착 판정이 깜빡이는 것을 막습니다.

관문 2 — 프레임 수와 시간, 둘 다

5 프레임 그리고 0.35 초를 모두 채워야 합니다. 프레임만 빨리 채우고 시간이 부족하면 통과하지 못합니다. 순간적으로 스쳐 지나간 것을 도착으로 오인하지 않기 위한 조건입니다.

도중에 outer(15 mm / 5 도)를 벗어나면 카운터가 0으로 초기화됩니다.

관문 3 — 속도 명령을 정확히 0으로

조건을 채우면 곧바로 도착이 아니라 ARRIVAL_WHEEL_STOP 상태로 가고, 속도 명령을 정확히 0으로 내립니다. 이 순간의 시각을 기록합니다.

관문 4 — 실제로 바퀴가 멈췄는지 확인 (가장 중요)

"0을 명령했다"는 것과 "멈췄다"는 것은 다릅니다. 관성, 슬립, 지연 때문에 명령 후에도 계속 밀려갑니다.

그래서 실측 바퀴 속도로 확인합니다.

- 직선 속도 $< 0.010 \text{ m/s}$ 그리고 회전 속도 $< 0.020 \text{ rad/s}$
- 이 상태가 0.35 초 유지
- 측정 시각이 0을 명령한 시각보다 나중이어야 함 — 명령 전에 측정된 "멈춘 상태"로는 통과 불가

- 매 측정이 직전보다 새로워야 함 — 같은 값 재사용 불가
- 측정 자체가 0.20 초 이내로 신선해야 함

관문 5 — 멈춘 자리가 여전히 맞는 자리인지

바퀴가 멈춘 그 순간에도 자세가 outer(15 mm / 5 도) 안에 있어야 합니다. 밀려서 벗어나 있으면 `arrival_cancelled_after_wheel_stop` 으로 도착을 취소하고 다시 정렬로 돌아갑니다.

다섯 관문을 모두 통과하면 ARRIVED_HOLD 입니다.

6. 전체 상태 흐름

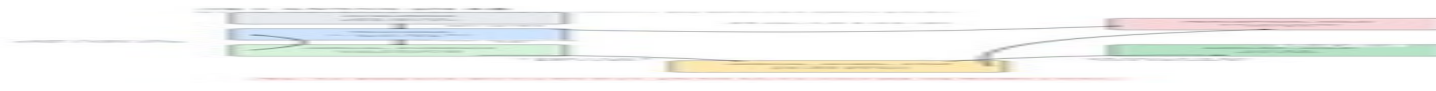


그림 3. 도착까지의 상태 전이와 각 관문.

7. 실행 로그 읽는 법

실제 실행에서 이런 줄이 나옵니다.

```
[pallet] frame=90 vision=fixed_outer_l_corner state=TRACKING dispatch=nonzero_proposal_accepted  
interlock=PASS  
[pallet] frame=120 vision=fixed_outer_l_corner state=ARRIVAL_WHEEL_STOP reason=wheel_stop_dwell  
interlock=PASS  
[pallet] frame=150 vision=fixed_outer_l_corner state=ARRIVED_HOLD reason=arrived_wheels_stopped  
interlock=PASS
```

필드	의미
vision=	지금 무엇을 보고 있는지. <code>unavailable</code> 이면 관측 실패로 정렬 불가
state=	서보 상태. 그림 3 의 박스 이름
dispatch=	<code>nonzero_proposal_accepted</code> 면 실제로 주행 중. <code>motion_interlock_selected_zero</code> 면 계산은 했지만 인터록이 0 으로 바꿈
interlock=	<code>PASS</code> 가 아니면 그 이유로 주행이 막힌 것. 상자 파지 상태와 상자 바닥-적층 간격을 검사하는 별도 게이트

8. 이 방식이 보장하는 것과 하지 않는 것

보장한다	보장하지 않는다
------	----------

재현성. 예전에 작업자가 맞춘 그 자리를 10 mm / 3 도 안으로 **절대 정확도.** 그 자리가 실제 슬롯 1 정중앙인지는 독립 측정이 다시 찾아간다 없다

안전한 정지. 명령이 아니라 실측 바퀴 속도로 정지를 확인한다 **배치 품질.** 10 mm 는 서보의 수렴 판정값이고, 적재 허용 오차 기준이 아니다

불확실할 때의 정지. 관측 상실·오래됨·점프·축 불일치에서 움직이지 않는다 **반복 성공률.** 몇 번 연속 성공해야 합격인지 기준이 아직 없다

불확실성이 두 겹입니다.

(1) 시연 기준점 자체가 눈대중입니다.

(2) 카메라와 로봇 base 사이의 등록(registration)도 nominal 이고, 경험적으로 얻은 y +50 mm 보정만

들어가 있습니다.

그래서 실기 자동 주행에는 --allow-nominal-registration 으로 이 위험을 명시적으로 승인해야

실행됩니다. 실제 정확도를 알려면 배치 후 상자 위치를 다시 측정해 목표와 비교해야 하고, 그 기능은

아직 구현되지 않았습니다.

9. 숫자 요약

항목	값	설정 키
목표 위치 (base)	[0.865000, 0.139523] m	slot1_hole_reference.center_base_xy_m
목표 방향	-90 도	slot1_hole_reference.yaw_base_deg
적층 외곽 (실측)	660 x 658 mm	outer_size_m
중앙 홀 (실측)	148 x 149 mm	opening_size_m
슬롯 1 오프셋	+0.20175 m (v_far), +0.128 m (u_right)	slot1_offset_*
위치 게인	0.7 /s	position_gain_per_s
yaw 게인	0.8 /s	yaw_gain_per_s
최대 직선 속도	0.05 m/s	max_linear_speed_mps
최대 회전 속도	0.08 rad/s	max_angular_speed_radps
중앙값 필터	3 프레임	filter_window (3 고정)
점프 거부	30 mm / 15 도	jump_threshold_m
관측 유효 기간	0.30 s	stale_after_s
보정량 한계	0.25 m	max_correction_m
도착 진입	10 mm / 3 도	arrival_inner_m, arrival_yaw_inner_rad
도착 유지	15 mm / 5 도	arrival_outer_m, arrival_yaw_outer_rad
도착 프레임 수	5	arrival_min_frames
도착 유지 시간	0.35 s	arrival_min_duration_s
바퀴 정지 판정	0.010 m/s, 0.020 rad/s	wheel_linear_stop_mps, wheel_angular_stop_radps
바퀴 정지 유지	0.35 s	wheel_stop_duration_s

10. 코드 위치

역할	파일
오차 → 속도 계산, 도착 판정	src/parcel_pose/pallet_servo.py
관측 소스 선택, 기준점 결합	src/parcel_pose/pallet_runtime.py (_servo_measurement)
적층 평면·홀·L 코너 인식	src/parcel_pose/pallet_geometry.py
전진 획득 권한 관리	src/parcel_pose/pallet_acquisition.py
기준점과 모든 수치	configs/rby1m_v1_2_pallet_slot1_nominal.json